Compilador da Simple Script Language

Saída do projeto com a etapa de Geração de Código (cap. 7 da apostila)
Rafael Schuinki e Mateus Mota · Compiladores – Prof. Luiz Guedes (IME)

1. Visão geral do pipeline

O compilador é um *front-end* de uma passada: léxico → sintático la-LR(1) dirigido por tabela → gramática de atributos (escopo, tipos e **geração de código**) → Linguagem de Montagem → Montador → Máquina Virtual de pilha (seção 7.1 da apostila).

`$ make clean && make`

```
gcc -std=c11 -Wall -Wextra -Wpedantic -o ssl main.c lexer.c grammar.c lalr.c parser.c scope.c
semantics.c codegen.c vm.c
gcc -std=c11 -Wall -Wextra -Wpedantic -o testes tests.c lexer.c grammar.c lalr.c parser.c scope.c
semantics.c codegen.c vm.c
```

`$ ./ssl --tabela | tail -3`

```
Terminais: 51  Nao terminais: 39  Estados la-LR(1): 175
Conflitos empilha/reduz: 1  reduz/reduz: 0
```

A tabela tem 88 regras (inclusive os marcadores semânticos ML, MT, MW, ME e MD da geração de código) e continua com apenas o conflito esperado do *else* pendente, resolvido por empilhar.

2. Exemplo 7.4.1 – fatorial.ssl

Código-fonte

```
function fatorial(n : integer) : integer
{
    var i, f : integer;
    i = 2;
    f = 1;
    while (i <= n)
        f = f * i++;
    return f;
}

function somaFat(n : integer) : integer
{
    var i, s : integer;
    i = 0;
    s = 0;
    while (++i <= n)
        s = s + fatorial(i);
    return s;
}
```

Linguagem de Montagem gerada

`$ ./ssl exemplos/fatorial.ssl`

```
BEGIN_FUNC 0 1 2                ; fatorial
        LOAD_CONST 0
        STORE_VAR 1
        LOAD_CONST 1
        STORE_VAR 2
L0:
        LOAD_VAR 1
        LOAD_VAR 0
        LE
        TJMP_FW L1
        LOAD_VAR 2
        LOAD_VAR 1
        DUP
        INC
        STORE_VAR 1
```

```
        MUL
        STORE_VAR 2
        JMP_BW L0
L1:
        LOAD_VAR 2
        RET
END_FUNC

BEGIN_FUNC 1 1 2               ; somaFat
        LOAD_CONST 2
        STORE_VAR 1
        LOAD_CONST 3
        STORE_VAR 2
L2:
        LOAD_VAR 1
        INC
        DUP
        STORE_VAR 1
        LOAD_VAR 0
        LE
        TJMP_FW L3
        LOAD_VAR 2
        LOAD_VAR 1
        CALL 0
        ADD
        STORE_VAR 2
        JMP_BW L2
L3:
        LOAD_VAR 2
        RET
END_FUNC

; Pool de Constantes:
;     0 = integer 2
;     1 = integer 1
;     2 = integer 0
;     3 = integer 0
```

Idêntico ao código da seção 7.4.1 da apostila (o único ponto em que o livro difere é o LT impresso no laço de `fatorial`, que no fonte é <=, ou seja, LE).

Programa montado (pools, endereços de byte e desvios relativos)

`$ ./ssl --montado exemplos/fatorial.ssl`

```
Pool de Funcoes:
  n   nome         params   vars     ip     retorno
  0   fatorial     1        2        0      1 celula(s)
  1   somaFat      1        2        33     1 celula(s)

Pool de Constantes:
    0 = integer 2
    1 = integer 1
    2 = integer 0
    3 = integer 0

Array de Codigo (68 bytes):
             ; ---- funcao fatorial ----
  0000  LOAD_CONST 0    ; integer 2
  0002  STORE_VAR  1
  0004  LOAD_CONST 1    ; integer 1
  0006  STORE_VAR  2
  0008  LOAD_VAR   1
  0010  LOAD_VAR   0
  0012  LE
  0013  TJMP_FW    17   ; -> 0030 (L1)
  0016  LOAD_VAR   2
  0018  LOAD_VAR   1
  0020  DUP
  0021  INC
  0022  STORE_VAR  1
  0024  MUL
  0025  STORE_VAR  2
  0027  JMP_BW     19   ; -> 0008 (L0)
```

```
  0030  LOAD_VAR   2
  0032  RET
              ; ---- funcao somaFat ----
  0033  LOAD_CONST 2    ; integer 0
  0035  STORE_VAR  1
  0037  LOAD_CONST 3    ; integer 0
  0039  STORE_VAR  2
  0041  LOAD_VAR   1
  0043  INC
  0044  DUP
  0045  STORE_VAR  1
  0047  LOAD_VAR   0
  0049  LE
  0050  TJMP_FW    15   ; -> 0065 (L3)
  0053  LOAD_VAR   2
  0055  LOAD_VAR   1
  0057  CALL       0    ; fatorial
  0059  ADD
  0060  STORE_VAR  2
  0062  JMP_BW     21   ; -> 0041 (L2)
  0065  LOAD_VAR   2
  0067  RET
```

Execução na máquina virtual

$ ./ssl --run exemplos/fatorial.ssl fatorial 5

```
fatorial(5) = 120   [58 instrucoes]
```

$ ./ssl --run exemplos/fatorial.ssl fatorial 10

```
fatorial(10) = 3628800   [118 instrucoes]
```

$ ./ssl --run exemplos/fatorial.ssl somaFat 4

```
somaFat(4) = 33   [177 instrucoes]
```

3. Exemplos 7.4.2 e 7.4.3 – strings.ssl

Código-fonte

```
function iguais(s1 : string, s2 : string, n : integer) : boolean
{
    var i : integer;
    i = 0;
    while (i < n && s1[i] == s2[i])
        i = i + 1;
    return i == n;
}

function inverte(s : string, n : integer) : string
{
    var i, j : integer;
    i = 0;
    j = n - 1;
    while (i < j) {
        var c : char;
        c = s[i];
        s[i] = s[j];
        s[j] = c;
        i = i + 1;
        j = j - 1;
    }
    return s;
}
```

$ ./ssl exemplos/strings.ssl

```
BEGIN_FUNC 0 3 1               ; iguais
        LOAD_CONST 0
        STORE_VAR 3
L0:
```

```
        LOAD_VAR 3
        LOAD_VAR 2
        LT
        LOAD_VAR 3
        CHAR_AT 0
        LOAD_VAR 3
        CHAR_AT 1
        EQ
        AND
        TJMP_FW L1
        LOAD_VAR 3
        LOAD_CONST 1
        ADD
        STORE_VAR 3
        JMP_BW L0
L1:
        LOAD_VAR 3
        LOAD_VAR 2
        EQ
        RET
END_FUNC

BEGIN_FUNC 1 2 3                ; inverte
        LOAD_CONST 2
        STORE_VAR 2
        LOAD_VAR 1
        LOAD_CONST 3
        SUB
        STORE_VAR 3
L2:
        LOAD_VAR 2
        LOAD_VAR 3
        LT
        TJMP_FW L3
        LOAD_VAR 2
        CHAR_AT 0
        STORE_VAR 4
        LOAD_VAR 2
        LOAD_VAR 3
        CHAR_AT 0
        SET_CHAR 0
        LOAD_VAR 3
        LOAD_VAR 4
        SET_CHAR 0
        LOAD_VAR 2
        LOAD_CONST 4
        ADD
        STORE_VAR 2
        LOAD_VAR 3
        LOAD_CONST 5
        SUB
        STORE_VAR 3
        JMP_BW L2
L3:
        LOAD_VAR 0
        RET
END_FUNC

; Pool de Constantes:
;     0 = integer 0
;     1 = integer 1
;     2 = integer 0
;     3 = integer 1
;     4 = integer 1
;     5 = integer 1
```

4. Exemplo 7.4.4 – ordena.ssl (bubble sort sobre array)

Código-fonte

```
type intArray = array [30] of integer

function bSort(a : intArray, n : integer) : intArray
```

```
{
    var i, j, v : integer;
    i = 0;
    while (i < n) {
        var trocou : boolean;
        trocou = false;
        j = i + 1;
        while (j < n) {
            var k : integer;
            if (a[i] > a[j]) {
                var t : integer;
                v = a[i];
                a[i] = a[j];
                a[j] = v;
                trocou = true;
            }
            k = j;
            j = k + 1;
        }
        i = i + 1;
    }
    return a;
}

function testeOrdena(n : integer) : integer
{
    var v : intArray;
    var i : integer;
    i = 0;
    while (i < 5) {
        var x : integer;
        x = (i * 7 + n) - (i * 7 + n) / 10 * 10;
        v[i] = x;
        i = i + 1;
    }
    v = bSort(v, 5);
    return v[0] * 10000 + v[1] * 1000 + v[2] * 100 + v[3] * 10 + v[4];
}
```

$ ./ssl exemplos/ordena.ssl

```
BEGIN_FUNC 0 31 6               ; bSort
        LOAD_CONST 1
        STORE_VAR 31
L0:
        LOAD_VAR 31
        LOAD_VAR 30
        LT
        TJMP_FW L1
        LOAD_FALSE
        STORE_VAR 34
        LOAD_VAR 31
        LOAD_CONST 2
        ADD
        STORE_VAR 32
L2:
        LOAD_VAR 32
        LOAD_VAR 30
        LT
        TJMP_FW L3
        LOAD_REF 0
        LOAD_VAR 31
        MUL 1
        ADD
        DE_REF 1
        LOAD_REF 0
        LOAD_VAR 32
        MUL 1
        ADD
        DE_REF 1
        GT
        TJMP_FW L4
        LOAD_REF 0
        LOAD_VAR 31
```

```
        MUL 1
        ADD
        DE_REF 1
        STORE_VAR 33
        LOAD_REF 0
        LOAD_VAR 31
        MUL 1
        ADD
        LOAD_REF 0
        LOAD_VAR 32
        MUL 1
        ADD
        DE_REF 1
        STORE_REF 1
        LOAD_REF 0
        LOAD_VAR 32
        MUL 1
        ADD
        LOAD_VAR 33
        STORE_REF 1
        LOAD_TRUE
        STORE_VAR 34
L4:
        LOAD_VAR 32
        STORE_VAR 35
        LOAD_VAR 35
        LOAD_CONST 3
        ADD
        STORE_VAR 32
        JMP_BW L2
L3:
        LOAD_VAR 31
        LOAD_CONST 4
        ADD
        STORE_VAR 31
        JMP_BW L0
L1:
        LOAD_REF 0
        DE_REF 30
        RET
END_FUNC

BEGIN_FUNC 1 1 32               ; testeOrdena
        LOAD_CONST 5
        STORE_VAR 31
L5:
        LOAD_VAR 31
        LOAD_CONST 6
        LT
        TJMP_FW L6
        LOAD_VAR 31
        LOAD_CONST 7
        MUL
        LOAD_VAR 0
        ADD
        LOAD_VAR 31
        LOAD_CONST 8
        MUL
        LOAD_VAR 0
        ADD
        LOAD_CONST 9
        DIV
        LOAD_CONST 10
        MUL
        SUB
        STORE_VAR 32
        LOAD_REF 1
        LOAD_VAR 31
        MUL 1
        ADD
        LOAD_VAR 32
        STORE_REF 1
        LOAD_VAR 31
        LOAD_CONST 11
        ADD
```

```
        STORE_VAR 31
        JMP_BW L5
L6:
        LOAD_REF 1
        LOAD_REF 1
        DE_REF 30
        LOAD_CONST 12
        CALL 0
        STORE_REF 30
        LOAD_REF 1
        LOAD_CONST 13
        MUL 1
        ADD
        DE_REF 1
        LOAD_CONST 14
        MUL
        LOAD_REF 1
        LOAD_CONST 15
        MUL 1
        ADD
        DE_REF 1
        LOAD_CONST 16
        MUL
        ADD
        LOAD_REF 1
        LOAD_CONST 17
        MUL 1
        ADD
        DE_REF 1
        LOAD_CONST 18
        MUL
        ADD
        LOAD_REF 1
        LOAD_CONST 19
        MUL 1
        ADD
        DE_REF 1
        LOAD_CONST 20
        MUL
        ADD
        LOAD_REF 1
        LOAD_CONST 21
        MUL 1
        ADD
        DE_REF 1
        ADD
        RET
END_FUNC

; Pool de Constantes:
;     1 = integer 0
;     2 = integer 1
;     3 = integer 1
;     4 = integer 1
;     5 = integer 0
;     6 = integer 5
;     7 = integer 7
;     8 = integer 7
;     9 = integer 10
;    10 = integer 10
;    11 = integer 1
;    12 = integer 5
;    13 = integer 0
;    14 = integer 10000
;    15 = integer 1
;    16 = integer 1000
;    17 = integer 2
;    18 = integer 100
;    19 = integer 3
;    20 = integer 10
;    21 = integer 4
```

5. estruturas.ssl – structs, arrays, ++/--, do-while, recursão

Código-fonte

```
type Ponto  = struct { x, y : integer }
type Reta   = struct { a : Ponto; b : Ponto; nome : string }
type Pontos = array [10] of Ponto
type Vetor  = array [5] of integer

function dist(p : Ponto, q : Ponto) : integer
{
    var dx, dy : integer;
    dx = p.x - q.x;
    dy = p.y - q.y;
    if (dx < 0) dx = -dx;
    if (dy < 0) dy = -dy;
    return dx + dy;
}

function fib(n : integer) : integer
{
    var r : integer;
    if (n <= 1) r = n; else r = fib(n - 1) + fib(n - 2);
    return r;
}

function perimetro(n : integer) : integer
{
    var v : Pontos;
    var i, total : integer;
    i = 0;
    while (i < 5) {
        var p : Ponto;
        p.x = i * n;
        p.y = i * i;
        v[i] = p;
        i = i + 1;
    }
    i = 0;
    total = 0;
    while (i < 4) {
        var d : integer;
        d = dist(v[i], v[i + 1]);
        total = total + d;
        i = i + 1;
    }
    return total;
}

function reta(n : integer) : integer
{
    var r : Reta;
    var k : integer;
    r.a.x = n;
    r.a.y = 1;
    r.b = r.a;
    r.b.x = r.b.x * 2;
    k = r.a.x++ + ++r.b.y;
    k = k * 100 + r.a.x * 10 + r.b.x;
    return k;
}

function rotulo(n : integer) : string
{
    var r : Reta;
    r.nome = "abc";
    r.nome[0] = 'X';
    r.nome[n] = r.nome[0];
    return r.nome + "!";
}

function laco(n : integer) : integer
{
```

```
    var i, s : integer;
    i = 0;
    s = 0;
    do {
        var t : integer;
        i = i + 1;
        if (i == 3) continue;
        if (i > n) break;
        t = i * i;
        s = s + t;
    } while (i < 100);
    return s;
}

function logica(a : integer, b : integer) : boolean
{
    var x : boolean;
    x = (a < b && !(a == 0)) || b == 7;
    return x;
}

function divmod(a : integer, b : integer) : integer
{
    var q : integer;
    q = a / b;
    return q * 100 + (a - q * b);
}

function ehDigito(c : char) : boolean
{
    var r : boolean;
    r = c >= '0' && c <= '9';
    return r;
}

function soma(v : Vetor, n : integer) : integer
{
    var i, s : integer;
    i = 0;
    s = 0;
    while (i < n) {
        var e : integer;
        e = v[i];
        s = s + e;
        i = i + 1;
    }
    return s;
}

function somaVet(n : integer) : integer
{
    var v : Vetor;
    var i : integer;
    i = 0;
    while (i < 5) {
        var d : integer;
        d = n + i;
        v[i] = d;
        i = i + 1;
    }
    return soma(v, 5);
}

function ponto(a : integer) : Ponto
{
    var p : Ponto;
    p.x = a;
    p.y = a * 2;
    return p;
}

function usaPonto(n : integer) : integer
{
    var p : Ponto;
    p = ponto(n);
```

```
    return p.x + p.y;
}
```

$ ./ssl exemplos/estruturas.ssl

```
BEGIN_FUNC 0 4 2                ; dist
        LOAD_REF 0
        ADD 0
        DE_REF 1
        LOAD_REF 2
        ADD 0
        DE_REF 1
        SUB
        STORE_VAR 4
        LOAD_REF 0
        ADD 1
        DE_REF 1
        LOAD_REF 2
        ADD 1
        DE_REF 1
        SUB
        STORE_VAR 5
        LOAD_VAR 4
        LOAD_CONST 2
        LT
        TJMP_FW L0
        LOAD_VAR 4
        NEG
        STORE_VAR 4
L0:
        LOAD_VAR 5
        LOAD_CONST 3
        LT
        TJMP_FW L1
        LOAD_VAR 5
        NEG
        STORE_VAR 5
L1:
        LOAD_VAR 4
        LOAD_VAR 5
        ADD
        RET
END_FUNC

BEGIN_FUNC 1 1 1                ; fib
        LOAD_VAR 0
        LOAD_CONST 4
        LE
        TJMP_FW L2
        LOAD_VAR 0
        STORE_VAR 1
        JMP_FW L3
L2:
        LOAD_VAR 0
        LOAD_CONST 5
        SUB
        CALL 1
        LOAD_VAR 0
        LOAD_CONST 6
        SUB
        CALL 1
        ADD
        STORE_VAR 1
L3:
        LOAD_VAR 1
        RET
END_FUNC

BEGIN_FUNC 2 1 25               ; perimetro
        LOAD_CONST 7
        STORE_VAR 21
L4:
        LOAD_VAR 21
        LOAD_CONST 8
```

```
        LT
        TJMP_FW L5
        LOAD_REF 23
        ADD 0
        LOAD_VAR 21
        LOAD_VAR 0
        MUL
        STORE_REF 1
        LOAD_REF 23
        ADD 1
        LOAD_VAR 21
        LOAD_VAR 21
        MUL
        STORE_REF 1
        LOAD_REF 1
        LOAD_VAR 21
        MUL 2
        ADD
        LOAD_REF 23
        DE_REF 2
        STORE_REF 2
        LOAD_VAR 21
        LOAD_CONST 9
        ADD
        STORE_VAR 21
        JMP_BW L4
L5:
        LOAD_CONST 10
        STORE_VAR 21
        LOAD_CONST 11
        STORE_VAR 22
L6:
        LOAD_VAR 21
        LOAD_CONST 12
        LT
        TJMP_FW L7
        LOAD_REF 1
        LOAD_VAR 21
        MUL 2
        ADD
        DE_REF 2
        LOAD_REF 1
        LOAD_VAR 21
        LOAD_CONST 13
        ADD
        MUL 2
        ADD
        DE_REF 2
        CALL 0
        STORE_VAR 25
        LOAD_VAR 22
        LOAD_VAR 25
        ADD
        STORE_VAR 22
        LOAD_VAR 21
        LOAD_CONST 14
        ADD
        STORE_VAR 21
        JMP_BW L6
L7:
        LOAD_VAR 22
        RET
END_FUNC

BEGIN_FUNC 3 1 6                ; reta
        LOAD_REF 1
        ADD 0
        ADD 0
        LOAD_VAR 0
        STORE_REF 1
        LOAD_REF 1
        ADD 0
        ADD 1
        LOAD_CONST 15
        STORE_REF 1
```

```
        LOAD_REF 1
        ADD 2
        LOAD_REF 1
        ADD 0
        DE_REF 2
        STORE_REF 2
        LOAD_REF 1
        ADD 2
        ADD 0
        LOAD_REF 1
        ADD 2
        ADD 0
        DE_REF 1
        LOAD_CONST 16
        MUL
        STORE_REF 1
        LOAD_REF 1
        ADD 0
        ADD 0
        DUP
        DUP
        DE_REF 1
        INC
        STORE_REF 1
        DE_REF 1
        DEC
        LOAD_REF 1
        ADD 2
        ADD 1
        DUP
        DUP
        DE_REF 1
        INC
        STORE_REF 1
        DE_REF 1
        ADD
        STORE_VAR 6
        LOAD_VAR 6
        LOAD_CONST 17
        MUL
        LOAD_REF 1
        ADD 0
        ADD 0
        DE_REF 1
        LOAD_CONST 18
        MUL
        ADD
        LOAD_REF 1
        ADD 2
        ADD 0
        DE_REF 1
        ADD
        STORE_VAR 6
        LOAD_VAR 6
        RET
END_FUNC

BEGIN_FUNC 4 1 5                ; rotulo
        LOAD_REF 1
        ADD 4
        LOAD_CONST 19
        STORE_REF 1
        LOAD_REF 1
        ADD 4
        LOAD_CONST 20
        LOAD_CONST 21
        CALL_LIB 0, 1
        LOAD_REF 1
        ADD 4
        LOAD_VAR 0
        LOAD_REF 1
        ADD 4
        LOAD_CONST 22
        CALL_LIB 0, 0
        CALL_LIB 0, 1
```

```
        LOAD_REF 1
        ADD 4
        DE_REF 1
        LOAD_CONST 23
        ADD
        RET
END_FUNC

BEGIN_FUNC 5 1 3                ; laco
        LOAD_CONST 24
        STORE_VAR 1
        LOAD_CONST 25
        STORE_VAR 2
L8:
        LOAD_VAR 1
        LOAD_CONST 26
        ADD
        STORE_VAR 1
        LOAD_VAR 1
        LOAD_CONST 27
        EQ
        TJMP_FW L10
        JMP_FW L11
L10:
        LOAD_VAR 1
        LOAD_VAR 0
        GT
        TJMP_FW L12
        JMP_FW L9
L12:
        LOAD_VAR 1
        LOAD_VAR 1
        MUL
        STORE_VAR 3
        LOAD_VAR 2
        LOAD_VAR 3
        ADD
        STORE_VAR 2
L11:
        LOAD_VAR 1
        LOAD_CONST 28
        LT
        NOT
        TJMP_BW L8
L9:
        LOAD_VAR 2
        RET
END_FUNC

BEGIN_FUNC 6 2 1                ; logica
        LOAD_VAR 0
        LOAD_VAR 1
        LT
        LOAD_VAR 0
        LOAD_CONST 29
        EQ
        NOT
        AND
        LOAD_VAR 1
        LOAD_CONST 30
        EQ
        OR
        STORE_VAR 2
        LOAD_VAR 2
        RET
END_FUNC

BEGIN_FUNC 7 2 1                ; divmod
        LOAD_VAR 0
        LOAD_VAR 1
        DIV
        STORE_VAR 2
        LOAD_VAR 2
        LOAD_CONST 31
        MUL
```

```
        LOAD_VAR 0
        LOAD_VAR 2
        LOAD_VAR 1
        MUL
        SUB
        ADD
        RET
END_FUNC

BEGIN_FUNC 8 1 1                ; ehDigito
        LOAD_VAR 0
        LOAD_CONST 32
        GE
        LOAD_VAR 0
        LOAD_CONST 33
        LE
        AND
        STORE_VAR 1
        LOAD_VAR 1
        RET
END_FUNC

BEGIN_FUNC 9 6 3                ; soma
        LOAD_CONST 34
        STORE_VAR 6
        LOAD_CONST 35
        STORE_VAR 7
L13:
        LOAD_VAR 6
        LOAD_VAR 5
        LT
        TJMP_FW L14
        LOAD_REF 0
        LOAD_VAR 6
        MUL 1
        ADD
        DE_REF 1
        STORE_VAR 8
        LOAD_VAR 7
        LOAD_VAR 8
        ADD
        STORE_VAR 7
        LOAD_VAR 6
        LOAD_CONST 36
        ADD
        STORE_VAR 6
        JMP_BW L13
L14:
        LOAD_VAR 7
        RET
END_FUNC

BEGIN_FUNC 10 1 7               ; somaVet
        LOAD_CONST 37
        STORE_VAR 6
L15:
        LOAD_VAR 6
        LOAD_CONST 38
        LT
        TJMP_FW L16
        LOAD_VAR 0
        LOAD_VAR 6
        ADD
        STORE_VAR 7
        LOAD_REF 1
        LOAD_VAR 6
        MUL 1
        ADD
        LOAD_VAR 7
        STORE_REF 1
        LOAD_VAR 6
        LOAD_CONST 39
        ADD
        STORE_VAR 6
        JMP_BW L15
```

```
L16:
        LOAD_REF 1
        DE_REF 5
        LOAD_CONST 40
        CALL 9
        RET
END_FUNC

BEGIN_FUNC 11 1 2               ; ponto
        LOAD_REF 1
        ADD 0
        LOAD_VAR 0
        STORE_REF 1
        LOAD_REF 1
        ADD 1
        LOAD_VAR 0
        LOAD_CONST 41
        MUL
        STORE_REF 1
        LOAD_REF 1
        DE_REF 2
        RET
END_FUNC

BEGIN_FUNC 12 1 2               ; usaPonto
        LOAD_REF 1
        LOAD_VAR 0
        CALL 11
        STORE_REF 2
        LOAD_REF 1
        ADD 0
        DE_REF 1
        LOAD_REF 1
        ADD 1
        DE_REF 1
        ADD
        RET
END_FUNC

; Pool de Constantes:
;     2 = integer 0
;     3 = integer 0
;     4 = integer 1
;     5 = integer 1
;     6 = integer 2
;     7 = integer 0
;     8 = integer 5
;     9 = integer 1
;    10 = integer 0
;    11 = integer 0
;    12 = integer 4
;    13 = integer 1
;    14 = integer 1
;    15 = integer 1
;    16 = integer 2
;    17 = integer 100
;    18 = integer 10
;    19 = string "abc"
;    20 = integer 0
;    21 = char 'X'
;    22 = integer 0
;    23 = string "!"
;    24 = integer 0
;    25 = integer 0
;    26 = integer 1
;    27 = integer 3
;    28 = integer 100
;    29 = integer 0
;    30 = integer 7
;    31 = integer 100
;    32 = char '0'
;    33 = char '9'
;    34 = integer 0
;    35 = integer 0
;    36 = integer 1
```

```
;    37 = integer 0
;    38 = integer 5
;    39 = integer 1
;    40 = integer 5
;    41 = integer 2
```

6. Execuções na máquina virtual

$ ./ssl --run exemplos/fatorial.ssl fatorial 6

```
fatorial(6) = 720   [70 instrucoes]
```

$ ./ssl --run exemplos/strings.ssl inverte abcdef 6

```
inverte("abcdef", 6) = "fedcba"   [81 instrucoes]
```

$ ./ssl --run exemplos/strings.ssl iguais abc abc 3

```
iguais("abc", "abc", 3) = true   [61 instrucoes]
```

$ ./ssl --run exemplos/strings.ssl iguais abc abd 3

```
iguais("abc", "abd", 3) = false   [46 instrucoes]
```

$ ./ssl --run exemplos/ordena.ssl testeOrdena 5

```
testeOrdena(5) = 23569   [659 instrucoes]
```

$ ./ssl --run exemplos/estruturas.ssl fib 15

```
fib(15) = 610   [24659 instrucoes]
```

$ ./ssl --run exemplos/estruturas.ssl perimetro 4

```
perimetro(4) = 32   [400 instrucoes]
```

$ ./ssl --run exemplos/estruturas.ssl reta 3

```
reta(3) = 546   [65 instrucoes]
```

$ ./ssl --run exemplos/estruturas.ssl rotulo 2

```
rotulo(2) = "XbX!"   [23 instrucoes]
```

$ ./ssl --run exemplos/estruturas.ssl laco 10

```
laco(10) = 376   [258 instrucoes]
```

$ ./ssl --run exemplos/estruturas.ssl logica 3 5

```
logica(3, 5) = true   [15 instrucoes]
```

$ ./ssl --run exemplos/estruturas.ssl divmod 17 5

```
divmod(17, 5) = 302   [14 instrucoes]
```

$ ./ssl --run exemplos/estruturas.ssl ehDigito 7

```
ehDigito('7') = true   [10 instrucoes]
```

$ ./ssl --run exemplos/estruturas.ssl somaVet 5

```
somaVet(5) = 35   [211 instrucoes]
```

$ ./ssl --run exemplos/estruturas.ssl ponto 9

```
ponto(9) = { 9, 18 }   [13 instrucoes]
```

$ ./ssl --run exemplos/estruturas.ssl usaPonto 4

```
usaPonto(4) = 12   [25 instrucoes]
```

$ ./ssl --run exemplos/estruturas.ssl divmod 1 0

```
Erro de execucao (ip = 551): divisao por zero
divmod(1, 0) =
```

$ ./ssl --run exemplos/estruturas.ssl rotulo 7

```
Erro de execucao (ip = 445): indice 7 fora do string
rotulo(7) =
```

7. Programas com erros (nenhum código é gerado)

$ ./ssl exemplos/erros_escopo.ssl

```
Erro de escopo (linha 1): identificador 'x' ja declarado neste escopo
Erro de escopo (linha 3): identificador 'a' ja declarado neste escopo
Erro de escopo (linha 6): identificador 'b' ja declarado neste escopo
Erro de escopo (linha 7): identificador 'c' nao declarado
Erro de escopo (linha 12): identificador 'g' nao declarado
Compilacao concluida com 5 erro(s).
```

$ ./ssl exemplos/erros_tipo.ssl

```
Erro de tipo (linha 9): 'n' nao e um tipo
Erro de tipo (linha 10): expressao booleana esperada
Erro de tipo (linha 11): tipos incompativeis
Erro de tipo (linha 12): tipo invalido para o operador
Erro de tipo (linha 13): acesso a campo em algo que nao e struct
Erro de tipo (linha 14): indexacao em algo que nao e array nem string
Erro de tipo (linha 15): indice deve ser do tipo integer
Erro de tipo (linha 16): campo 'z' nao declarado na struct
Erro de tipo (linha 17): 'Ponto' nao e variavel nem parametro
Erro de tipo (linha 18): 'n' nao e funcao
Erro de tipo (linha 19): argumentos demais na chamada
Erro de tipo (linha 20): argumentos de menos na chamada
Erro de tipo (linha 21): tipo do argumento 1 incompativel com o parametro
Erro de tipo (linha 22): tipos incompativeis
Erro de tipo (linha 23): tipo invalido para o operador
Erro de tipo (linha 24): tipo invalido para o operador
Erro de tipo (linha 25): expressao booleana esperada
Erro de tipo (linha 26): tipos incompativeis
Erro de tipo (linha 27): tipo do return incompativel com o retorno da funcao
Erro semantico (linha 33): break/continue fora de comando de repeticao
Compilacao concluida com 20 erro(s).
```

$ ./ssl exemplos/erro_sintatico.ssl

```
Erro sintatico (linha 4): encontrado ';'; esperado: 'false', 'true', '(', '++', '-', '--', '!',
caractere, numero, string, identificador
Compilacao concluida com 1 erro(s).
```

$ ./ssl exemplos/erro_lexico.ssl

```
Erro lexico (linha 4): caractere invalido '#' (0x23)
Erro lexico (linha 5): '&' isolado (esperado '&&')
Compilacao concluida com 2 erro(s).
```

8. Suíte de testes

$ ./testes

```
Erro lexico (linha 6): caractere invalido '#' (0x23)
OK: 85 testes
```